

INSTITUTO TECNOLÓGICO DE CULIACAN



Mauricio Lopez Lara

Carlos Ley Garcia

09/05/2026

Tópicos avanzados de IA

Tarea Bonus algoritmos

El problema del agente viajero resuelto a base de Algoritmos Genéticos

El problema del agente viajero es uno de los problemas que se encarga de la optimización mezclada que son de los mas estudiados en la investigación de operaciones y la informática teórica, el objetivo es realmente simple: encontrar la ruta mas corta posible que visite un conjunto de ciudades exactamente una vez y este regrese a su punto de origen.

En cuanto a su complejidad esto se mantiene como un NP-HARD, esto significa que a medida que aumenta el número de ciudades, entonces las posibles permutaciones de la ruta crecen factorial mente esto quiere decir que se expresara con números que se puedan multiplicar o dicho de otro modo números primos, jen este caso serian 20 ciudades lo cual seria $19! / 2$, lo que equivale a aproximadamente 60 trillones de combinaciones y debemos resolver todas estas rutas.

Para poder repartir esta explosión de combinaciones, en este caso usaremos la heurística y la metaheurística, los algoritmos genéticos no garantizan encontrar una solución perfecta absoluta, pero son altamente eficientes para poder encontrar soluciones lo suficientemente buenas en tiempos de procesamiento extremadamente reducido.

Aplicación Practica

La resolución eficiente de esta matriz de distancias de 20 ciudades tiene un impacto directo y cuantificable en operaciones logísticas.

Si asumiéramos que la matriz representa costos de distancias en kilómetros, la ruta generada por el Algoritmo Genético dictara la forma mas eficiente para que un grupo de camiones de reparto entregue mercancía desde un almacén central a 19 sucursales en todo el país. La minimización de la distancia se traduce directamente en ahorro de combustible, reducción de desgaste del grupo y la disminución de emisiones del CO₂.

Para el mantenimiento un equipo de técnicos visitase todas las sedes que se pueden utilizar o que usaran la ruta generada para minimizar los viáticos y el tiempo inactivo viajando.

En el caso de la fabricación la lógica es conectar los 20 puntos de la forma más rápida posible es la misma que utiliza las maquinas perforadoras industriales para hacer agujeros en las placas base de computadoras con el objetivo de optimizar el tiempo de movimiento del taladro.

Diseño del algoritmo genético

1. Representación del genoma

Se utilizó una codificación de permutaciones de enteros, esto porque cada ciudad fue asignada a un índice por mencionar algunos: Madrid=0, Barcelona=1 y así con el resto de las ciudades hasta el 19 que es Elche.

El individuo será la lista que representa el orden de visita válido y la población inicial serán de 100 rutas generadas de forma completamente aleatoria para asegurar la diversidad.

2. Función de Aptitud

Esta función es llamada fitness y está dentro de un individuo como el costo total de su ruta entre las ciudades contiguas, más el regreso a la ciudad inicial. Como queremos una ruta más corta, el objetivo del algoritmo es minimizar este valor en términos de selección, una ruta con un costo menor tiene un fitness mayor o dicho de otro modo es la más atractiva para la reproducción.

3. Operadores Genéticos

- Se eligen 3 individuos al azar y el que tenga la ruta más corta gana el derecho a reproducirse porque esto asegura que las buenas soluciones prosperen pero permite que algunas rutas malas sobrevivan pero de manera ocasional, evitando caer en un óptimo local prematuro.
- Un cruce normal podría generar hijos con ciudades repetidas o ciudades faltantes, el cruce de orden copia un segmento de la ruta padre A al hijo y rellena los huecos restantes con las ciudades de la ruta padre B en el orden que aparecen, garantizando rutas válidas.
- Con una probabilidad baja por decir un porcentaje digamos que un 5% puede que dos ciudades elegidas al azar en la ruta de un hijo intercambian sus posiciones. La mutación es vital para introducir una mutación a la población y explorar nuevas áreas de la matriz que el cruce no alcanzaría por sí solo.

4. Desarrollo de la solución en Python

Para la implementación, se consideró usar una librería preconstruida como DEAP, pero al final se construyó un algoritmo desde cero utilizando únicamente las librerías estándar de Python.

Esto nos permitió un control absoluto sobre los operadores de cruce y una lectura transparente de la lógica paso a paso y si hablamos de desventajas de esto es que puede ser ligeramente menos veloz en ejecución pura comprobando con librerías compiladas en C pero como estamos usando 20

ciudades el tiempo de calculo sigue siendo de pocos segundo lo cual seamos honestos es altamente aceptable para el problema que se nos planteó.

Código en Python:

```
Import random
```

```
Nombres_ciudades = ["Madrid", "Barcelona", "Valencia", "Sevilla", "Zaragoza",  
"Málaga", "Murcia", "Palma", "Las Palmas", "Bilbao", "Alicante", "Córdoba",  
"Valladolid", "Vigo", "Gijón", "L'Hospitalet", "Vitoria", "A Coruña", "Granada",  
"Elche"]
```

```
Matriz_distancias = [[0, 303, 442, 223, 252, 654, 737, 458, 555, 279, 726, 456,  
876, 467, 614, 249, 709, 368, 435, 530],[303, 0, 409, 591, 341, 403, 263, 485,  
385, 671, 595, 738, 591, 725, 502, 328, 527, 617, 144, 244],[442, 409, 0, 430,  
273, 639, 471, 152, 709, 335, 178, 574, 383, 281, 387, 624, 310, 480, 240,  
683],[223, 591, 430, 0, 619, 431, 617, 629, 754, 124, 534, 630, 713, 295, 491,  
245, 630, 295, 501, 478],[252, 341, 273, 619, 0, 332, 606, 203, 506, 470, 250,  
562, 563, 334, 311, 760, 111, 757, 631, 349],[654, 403, 639, 431, 332, 0, 392,  
306, 432, 606, 539, 401, 234, 449, 709, 520, 317, 192, 549, 271],[737, 263,  
471, 617, 606, 392, 0, 854, 801, 89, 617, 390, 698, 595, 572, 405, 183, 164,  
360, 337], [458, 485, 152, 629, 203, 306, 854, 0, 390, 323, 733, 391, 440, 319,  
252, 735, 394, 408, 443, 456], [555, 385, 709, 754, 506, 432, 801, 390, 0, 671,  
395, 200, 684, 206, 423, 792, 423, 614, 241, 185], [279, 671, 335, 124, 470,  
606, 89, 323, 671, 0, 305, 534, 406, 842, 475, 516, 778, 579, 483, 864], [726,  
595, 178, 534, 250, 539, 617, 733, 395, 305, 0, 767, 130, 551, 463, 554, 432,  
593, 529, 864], [456, 738, 574, 630, 562, 401, 390, 391, 200, 534, 767, 0, 517,  
853, 205, 534, 261, 380, 489, 515],[876, 591, 383, 713, 563, 234, 698, 440,  
684, 406, 130, 517, 0, 387, 558, 495, 308, 557, 590, 443], [467, 725, 281, 295,  
334, 449, 595, 319, 206, 842, 551, 853, 387, 0, 183, 171, 438, 445, 417, 678],  
[614, 502, 387, 491, 311, 709, 572, 252, 423, 475, 463, 205, 558, 183, 0, 527,  
477, 294, 625, 200],[249, 328, 624, 245, 760, 520, 405, 735, 792, 516, 554,  
534, 495, 171, 527, 0, 466, 664, 188, 437],[709, 527, 310, 630, 111, 317, 183,  
394, 423, 778, 432, 261, 308, 438, 477, 466, 0, 761, 655, 752],[368, 617, 480,  
295, 757, 192, 164, 408, 614, 579, 593, 380, 557, 445, 294, 664, 761, 0, 369,  
634],[435, 144, 240, 501, 631, 549, 360, 443, 241, 483, 529, 489, 590, 417,  
625, 188, 655, 369, 0, 798], [530, 244, 683, 478, 349, 271, 337, 456, 185, 864,  
864, 515, 443, 678, 200, 437, 752, 634, 798, 0] ]
```

```
NUM_CIUDADES=20
```

```
TAMAÑO_POBLACION = 100
```

```
GENERACIONES = 1000
```

```
PROBABILIDAD_MUTACION= 0.05
```

```

Def calcular_distancia_ruta(ruta):
    Distancia_total = 0
    For i in range(len(ruta)):
        Origen = ruta[i]
        Destino = ruta (i + 1) % len (ruta)]
        Distancia_actual += matriz_distancias[origen][destino]
    Return distancia_total

Def crear_poblacion_inicial():
    Población = []
    For _ in range (TAMAÑO_POBLACION):
        Ruta = list(range(NUM_CIUDADES))
        Random_shuffle(ruta)
        Población.append(ruta)
    Return población

Def selección_torneo(poblacion):
    Participantes = random.sample(población, 3)
    Mejor = min(participante, key=calcular_distancia_ruta)
    Return mejor

Def cruce_orden(padre1, padre 2):
    Inicio, fin = sorted(random.sample(range(NUM_CIUDADES),2))
    Hijo = [-1] * NUM_CIUDADES
    Hijo[inicio:fin+1] = padre1[inicio:fin+1]
    Pos_padre2 = 0
    For i in range(NUM_CIUDADES):
        If hijo[i] == -1:
            While padre2[pos_padre2] in hijo:
                Pos_padre2 += 1
            Hijo[i] = padre2[pos_padre2]

```

```

Return hijo
Def mutar(ruta)
If random.random () < PROBABILIDAD_MUTACION
Idx1, idx2 = random.sample (sample(NUM_CIUDADES),2)
Ruta[idx1] , Ruta[idx2] = Ruta[idx2] , Ruta[idx1]
Población_actual = crear_poblacion_inicial ()
mejor_ruta_historica = None
mejor_distancia_historica = float ('inf')
for generación in range(GENERACIONES):
nueva_poblacion = []
mejor_actual = min (población_actual, key=calcular_distancia_ruta)
distancia_actual = calcular_distancia_ruta(mejor_actual)
if distancia_actual < mejor_distancia_historica:
mejor_distancia_historica = distancia_actual
mejor_ruta_historica = list(mejor_actual)
nueva_poblacion.append(mejor_actual)
while len(nueva_poblacion) < TAMAÑO_POBLACION:
padre1 = selección_torneo(población_actual)
padre2 = selección_torneo(población_actual)
hijo = cruce_orden(padre1, padre2)
mutar(hijo)
nueva_poblacion.append(hijo)
población_actual = nueva_poblacion
nombres_ruta = [nombres_ciudades[i] for i in mejor_ruta_historica]
nombres_ruta.append(nombres_ruta[0])
print(f"Mejor ruta encontrada: {' -> '.join(nombres_ruta)}")
print(f"Distancia Total Óptima Calculada: {mejor_distancia_historica}")

```

Este código al ser ejecutado sobre la matriz 20*20 el algoritmo inicializa con rutas caóticas de costos altos pero a través de 1000 generaciones la selección de torneo garantiza que las subsecuencias lógicas sobrevivan y se combinen, el resultado arrojado por el programa no es solo un vector numerico, sino una secuencia estratégica de despliegue que minimizar el consumo de recursos de la empresa logística, demostrando la superioridad de las biomimesis computacional frente al análisis tradicional

<https://github.com/LeyCarlos-21170368/Topicos-de-IA/tree/main/Módulo%203>